

머신러닝 및 실습

(Lecture 9)

노연수

(esnoh@koreatech.ac.kr)

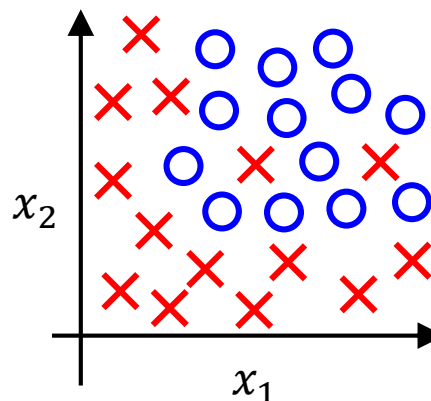
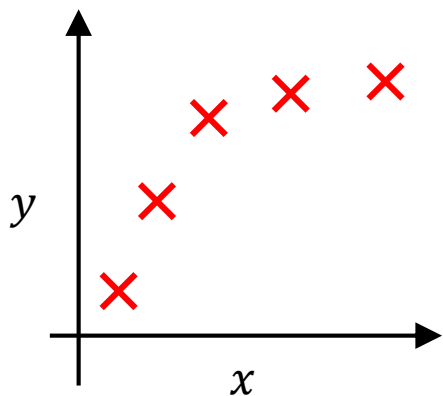
Course Plan

Week	Lecture	Contents	Notes
1	-	-	대체휴일
2	파이썬 기초 I	자료형, 연산자, 조건문, 반복문	
3	파이썬 기초 II	함수, 모듈, 패키지, 클래스	
4	머신러닝이란 무엇인가?	머신러닝 기초 개념, 머신러닝 시스템의 종류 등	
5	머신러닝의 전체적인 흐름 이해하기 I	프로젝트 방향 수립, 데이터 수집, 데이터 분석	
6	머신러닝의 전체적인 흐름 이해하기 II	데이터 전처리, 모델 학습, 모델 튜닝	
7	분류모델의 이해	이진 분류기 훈련, 성능 측정, 다중 분류	
8	중간고사	-	
9	회귀모델의 이해	선형 회귀, 경사 하강법	프로젝트 공지
10	회귀모델의 확장	다항 회귀, 로지스틱 회귀	
11	서포트 벡터 머신의 이해	규제 선형 모델, SVM 분류/회귀 등	
12	다양한 분류 모델과 앙상블	결정 트리, 랜덤 포레스트	
13	차원 축소	차원 축소, 주성분 분석(PCA)	정상수업(5/25)
14	군집 분석	K-Means clustering	
15	프로젝트 발표	프로젝트 발표	
16	프로젝트 발표, 기말고사	기말고사	

과적합(overfitting)

▶ 모델이 훈련 데이터에 지나치게 맞춰져 새로운 데이터에 일반화되지 못하는 현상

- 특징 개수가 많을 경우
- 데이터에 비해 예측 모델이 복잡할 경우(예: 고차 다항 모델) 등

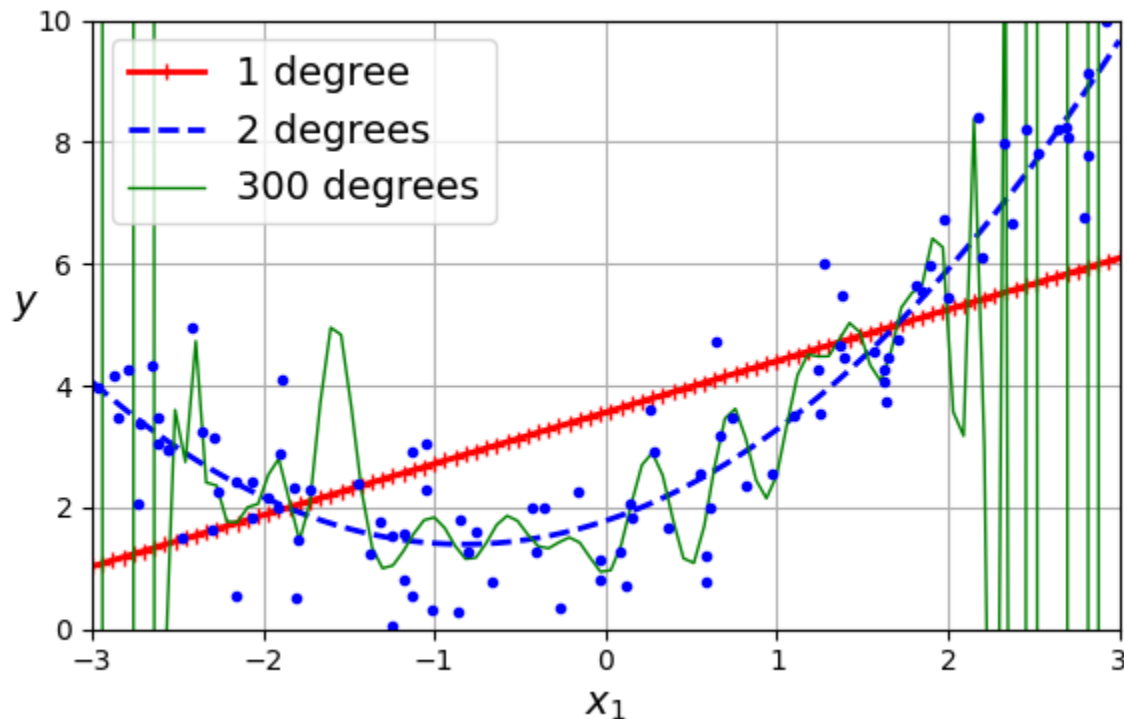


규제(Regularization)

■ 규제의 필요성

▶ (회귀, 분류) 모델 훈련 중 과적합(overfitting)을 방지하기 위해 필요

- 특징 개수를 줄임
- 예측 모델을 단순화(저차 다항 모델)
- 규제(Regularization)



규제 선형 모델(Regularized Linear Model)

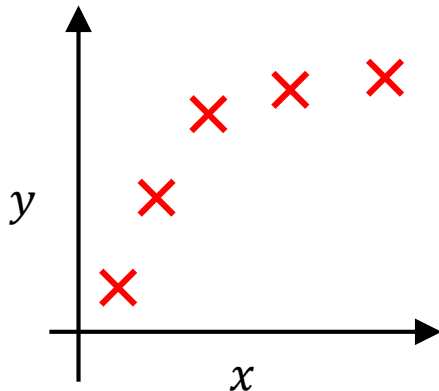
■ 규제 선형 모델

▶ **훈련 시** 비용(손실)함수에 규제항을 추가하여 모델의 과적합을 완화하는 선형 모델

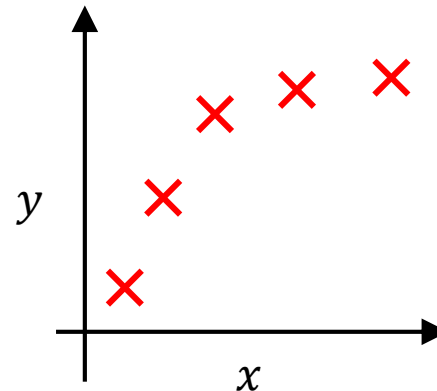
- 가중치 ↓ → 예측 모델 단순화 → 과적합 확률 ↓

$$\min \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 1000 \theta_3^2 + 1000 \theta_4^2$$

$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$



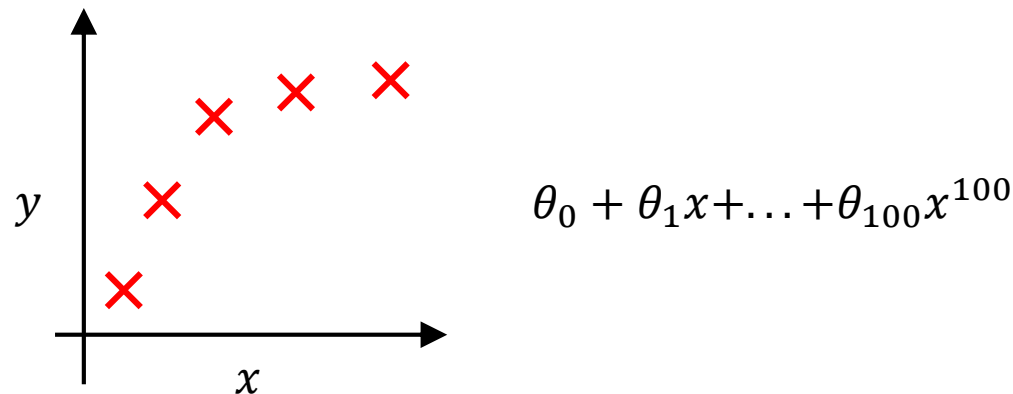
규제 선형 모델(Regularized Linear Model)

■ 규제 선형 모델의 비용 함수(cost function)

▶ 규제 파라미터(α , regularization parameter)를 조절하여 규제의 정도를 조절 가능

- (주의) 편향(bias)은 규제되지 않음
- 손실항과 규제항은 트레이드오프(trade-off) 관계
- 만약 α 가 극단적으로 크면?

$$J(\theta) = \frac{1}{n} \left[\underbrace{\sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{예측값-실제값 손실항}} + \alpha \underbrace{\sum_{j=1}^m \theta_j^2}_{\text{규제항}} \right]$$



규제 선형 모델(Regularized Linear Model)

Gradient descent

- ▶ 규제항에 대한 편도함수(partial derivative)를 구하여 함께 업데이트
 - (주의) 편향은 규제되지 않음

$\theta_0, \dots, \theta_m$ 를 임의의 값으로 초기화

수렴 조건을 만족할 때 까지 반복 {

$$\theta_0 := \theta_0 - \eta \frac{2}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_j := \theta_j - \eta \left[\frac{2}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{2\alpha}{n} \theta_j \right] \quad (j = 1, 2, \dots, n)$$

}

$J(\theta)$ 를 최소화 하는 $\theta_0, \dots, \theta_m$ 선택

규제 선형 모델(Regularized Linear Model)

릿지 회귀(Ridge regression)

▶가중치의 제곱합을 규제항으로 추가한 선형 회귀 모델

- 훈련 중 손실항을 최소화하면서 가중치의 크기도 작게 유지하도록 함

— <릿지 회귀 비용 함수> —

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{n} \sum_{j=1}^m \theta_j^2 = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha \|\mathbf{w}\|_2^2}{n}$$

— <참고> —

$$\text{MSE}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

$$\mathbf{w} = [\theta_1, \theta_2, \dots, \theta_m]$$

$$\|\mathbf{w}\|_2 = \sqrt{\theta_1^2 + \theta_2^2 + \dots + \theta_m^2} \rightarrow \ell_2 \text{ 노름(norm)}$$

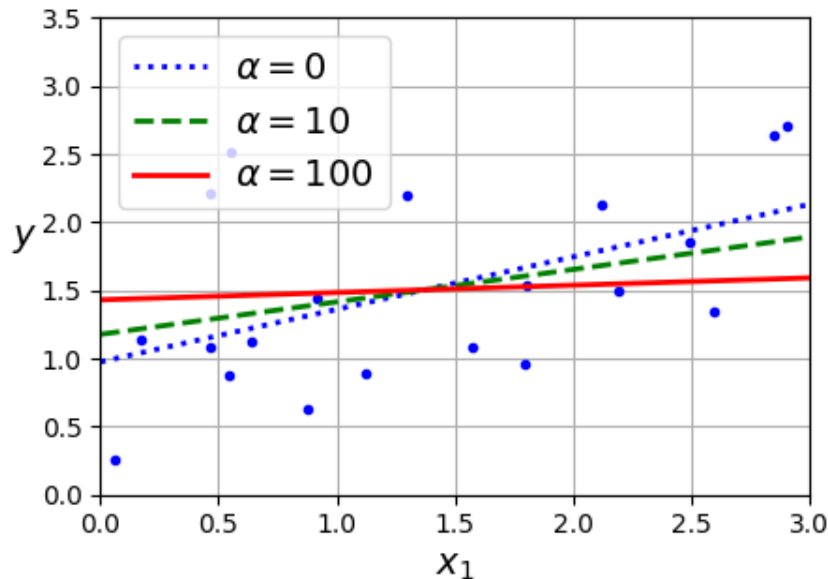
규제 선형 모델(Regularized Linear Model)

릿지 회귀(Ridge regression) 특징

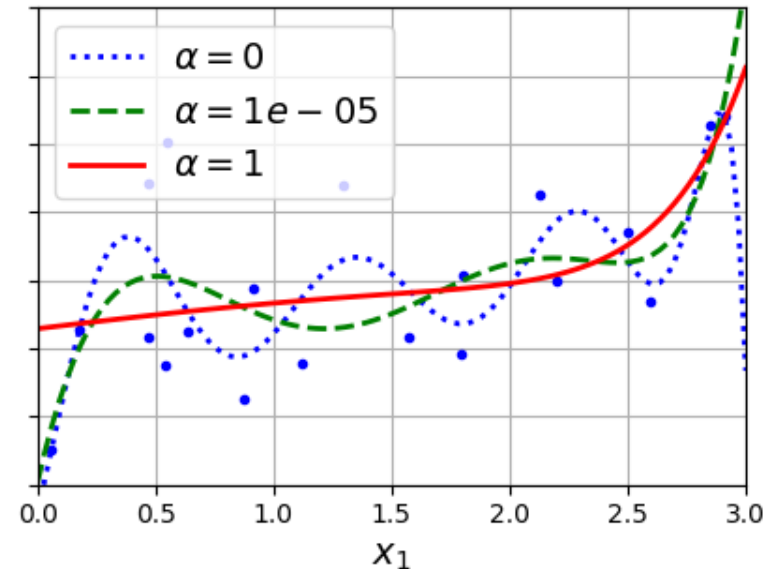
▶ 규제 파라미터(α)가 증가할수록 직선에 가까워짐

- $\alpha = 0$: 선형 회귀
- $\alpha \gg 0$: 수평선

<선형 회귀>



<다항 회귀(degree=10)>



규제 선형 모델(Regularized Linear Model)

■ 라쏘 회귀(Lasso regression)

▶ 가중치의 절댓값 합을 규제항으로 추가한 선형 회귀 모델

- 훈련 중 일부 가중치를 0으로 만들어 중요한 특징만 선택하도록 함

————— <릿지 회귀 비용 함수> —————

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{j=1}^m |\theta_j| = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \|\mathbf{w}\|_1$$

————— <참고> —————

$$\mathbf{w} = [\theta_1, \theta_2, \dots, \theta_m]$$

$$\|\mathbf{w}\|_1 = |\theta_1| + |\theta_2| + \dots + |\theta_m| \rightarrow \ell_1 \text{ 노름(norm)}$$

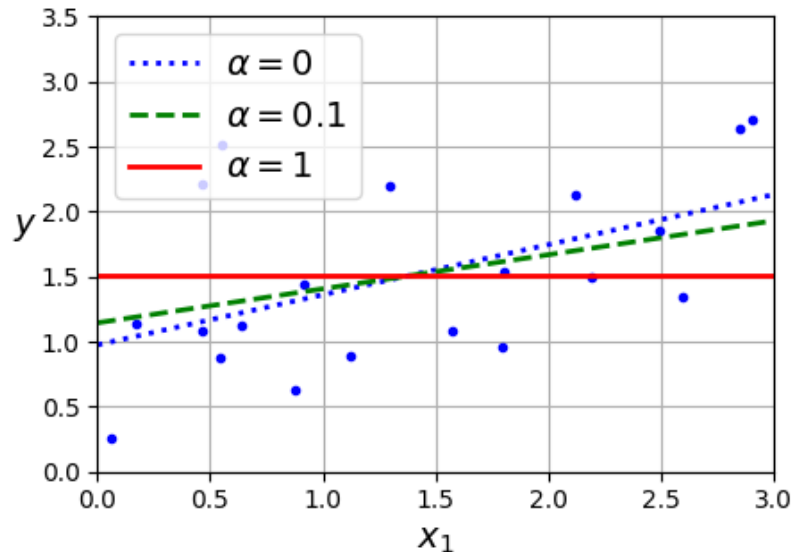
규제 선형 모델(Regularized Linear Model)

라쏘 회귀(Lasso regression) 특징

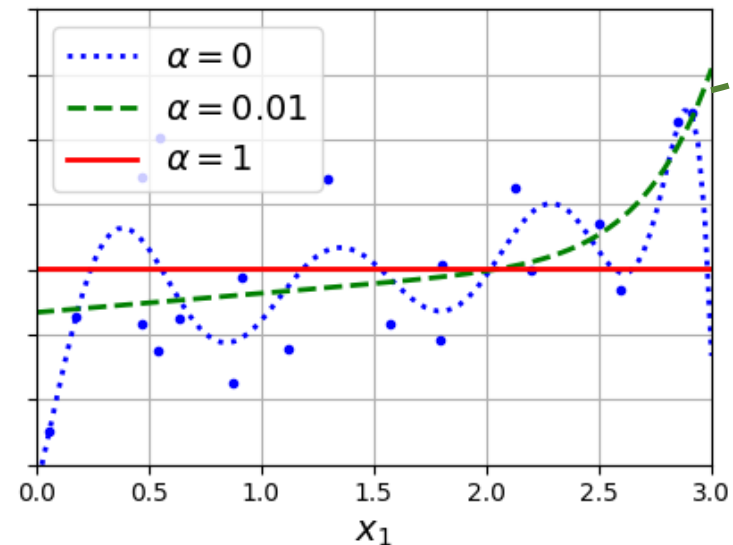
▶ 훈련 중 일부 가중치를 0으로 만들어 중요한 특징만 선택하도록 함

- $\alpha = 1$: 수평선

<선형 회귀>



<다항 회귀(degree=10)>



고차항 가중치(4~10)를 0으로 만들

규제 선형 모델(Regularized Linear Model)

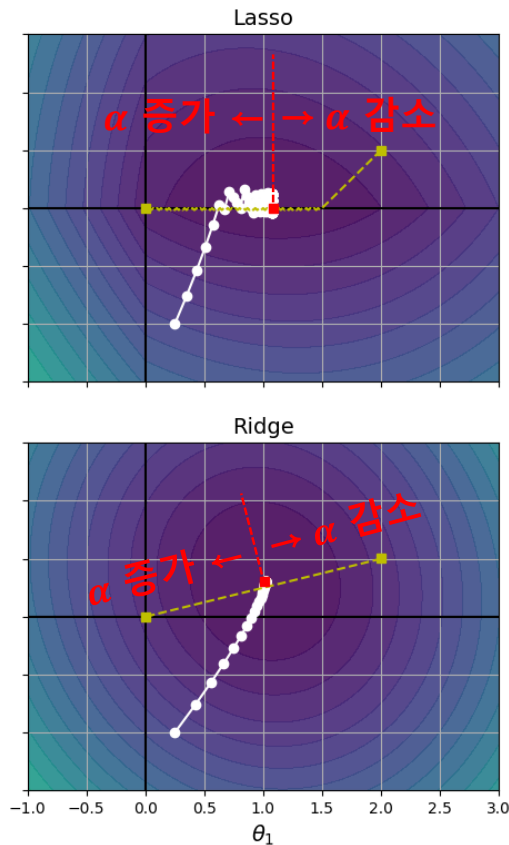
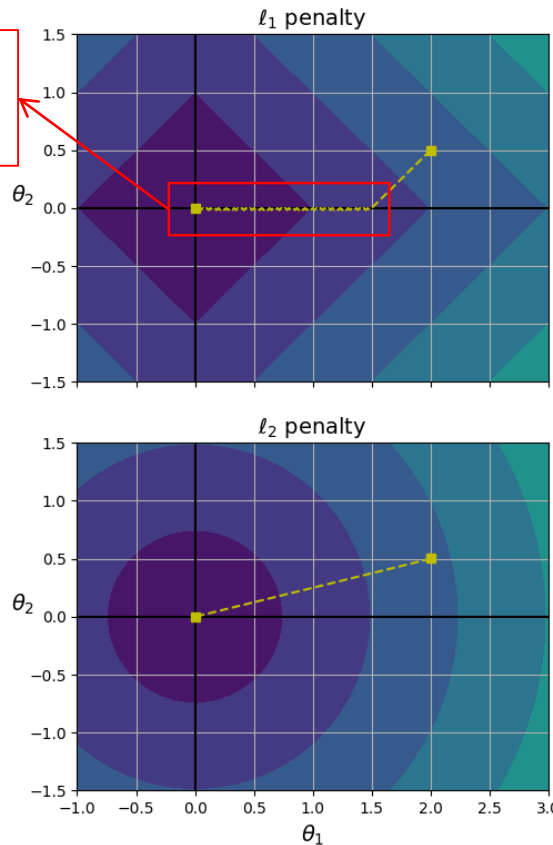
Lasso와 Ridge

- ▶ Lasso 회귀: 일부 가중치를 0으로 만들어 중요한 특징만 남김
- ▶ Ridge 회귀: 가중치를 0에 가깝게 줄이지만, 0으로 만들지는 않음

<규제 손실>

<모델 별 비용함수>

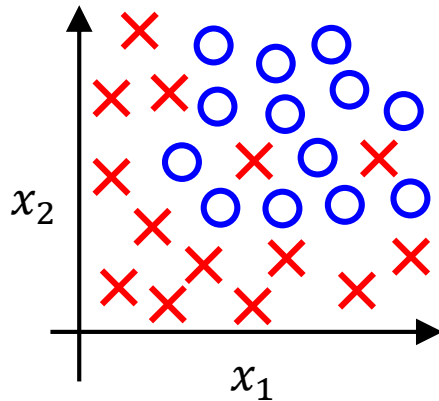
규제항의 그래디언트가
0에서 정의되지 않기
때문에 진동함



규제 로지스틱 회귀(Regularized Logistic Regression)

개념 및 비용 함수

▶ 규제 선형 모델과 전체적인 개념이 동일



$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

$$J(\theta) = -\frac{1}{n} \left(\sum_{i=1}^n [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})] + \alpha \sum_{j=1}^m \theta_j^2 \right)$$

규제 로지스틱 회귀(Regularized Logistic Regression)

Gradient descent

▶ 규제 선형 모델과 전체적인 개념이 동일

$\theta_0, \dots, \theta_m$ 를 임의의 값으로 초기화

수렴 조건을 만족할 때 까지 반복 {

$$\theta_0 := \theta_0 - \eta \frac{2}{n} \sum_{i=1}^n (\sigma(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) - y^{(i)})$$

$$\theta_j := \theta_j - \eta \left[\frac{2}{n} \sum_{i=1}^n (\sigma(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{2\alpha}{n} \theta_j \right] \quad (j = 1, 2, \dots, m)$$

}

$J(\boldsymbol{\theta})$ 를 최소화 하는 $\theta_0, \dots, \theta_m$ 선택

■ 엘라스틱 넷 회귀(Elastic net regression)

▶ Lasso 회귀와 Ridge 회귀의 규제를 함께 사용하는 선형 회귀 모델

- Lasso: 일부 가중치를 0으로 만들어 특징 선택 효과
- Ridge: 가중치를 작게 만듦
- Elastic net: 손실 줄이면서, 가중치 작게 유지 + 일부 가중치를 0으로 만들어 특징 선택

— <엘라스틱 넷 회귀 비용 함수> —

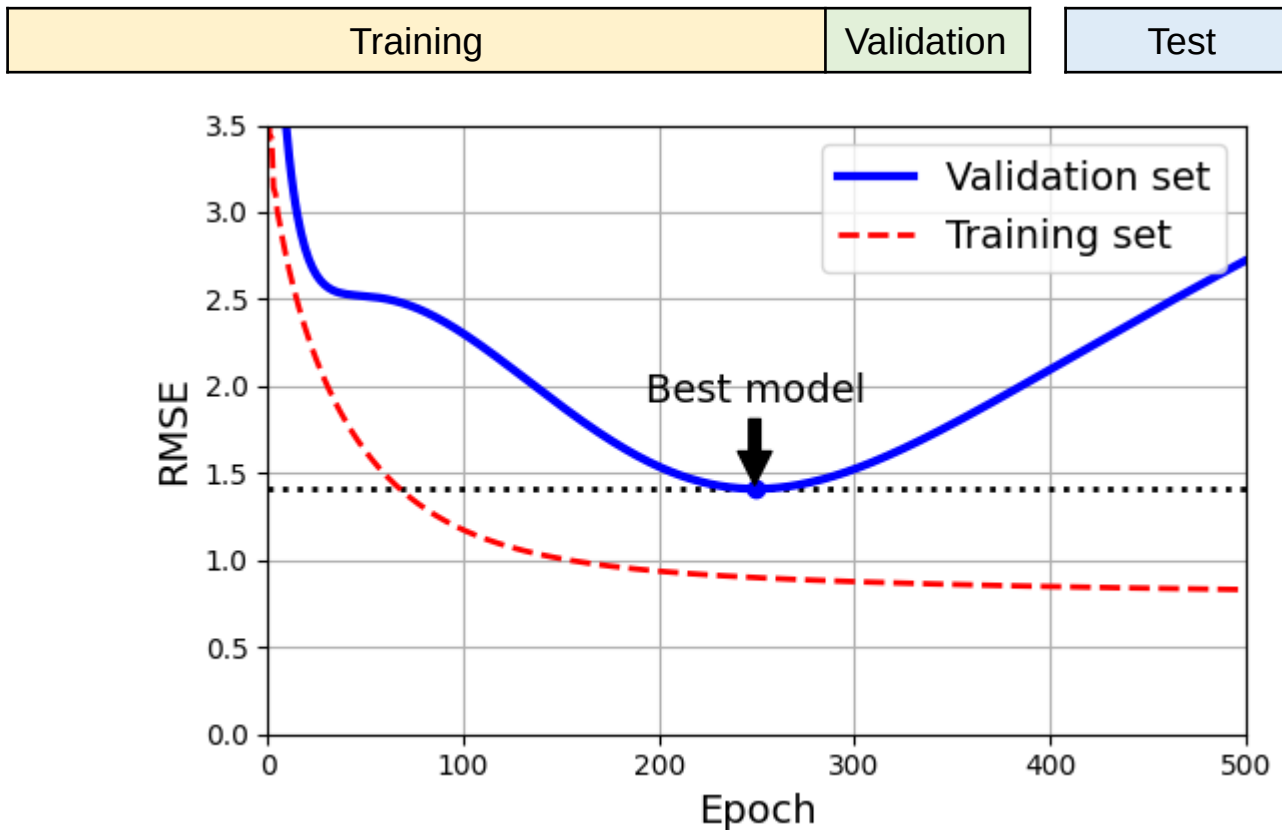
$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r \left(2\alpha \sum_{j=1}^m |\theta_j| \right) + (1-r) \left(\frac{\alpha}{n} \sum_{j=1}^m \theta_j^2 \right)$$

조기 종료(Early Stopping)

조기 종료(Early Stopping)

▶ 경사 하강법과 같은 반복적인 학습 알고리즘을 규제하는 방법

- 검증 오차(validation error)가 최솟값에 도달하면 강제로 훈련을 중지
- 검증 오차가 증가하면 모델이 과적합 되었다는 의미



규제 선형 모델(Regularized Linear Model)

정리

구 분	사용하는 경우
선형 회귀	· 규제한 모델로 쓰세요^^
Ridge	· 기본
Lasso	· 일부 특징만 중요한 경우
Elastic net	· 일부 특징만 중요한 경우 · 특징간 상관관계가 높은 경우 · 특징 수 > 훈련 샘플 수

상황에 따라 모델 간 성능을 비교해볼 필요가 있습니다

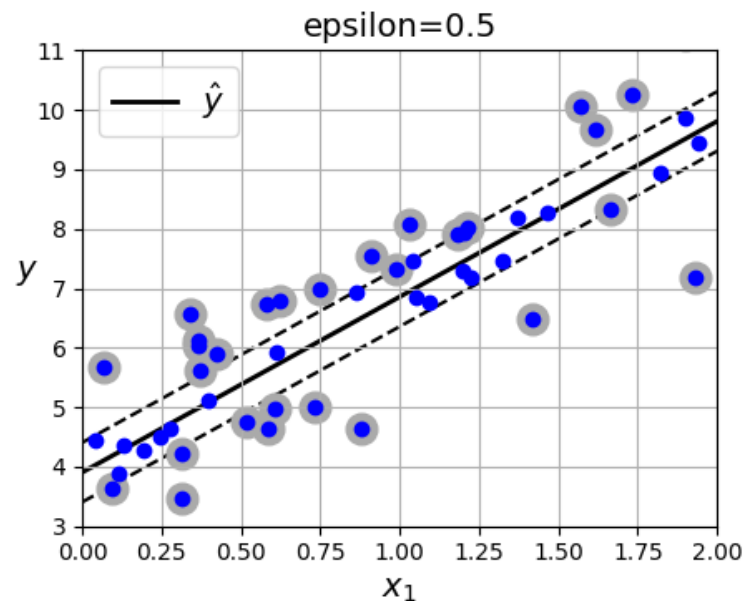
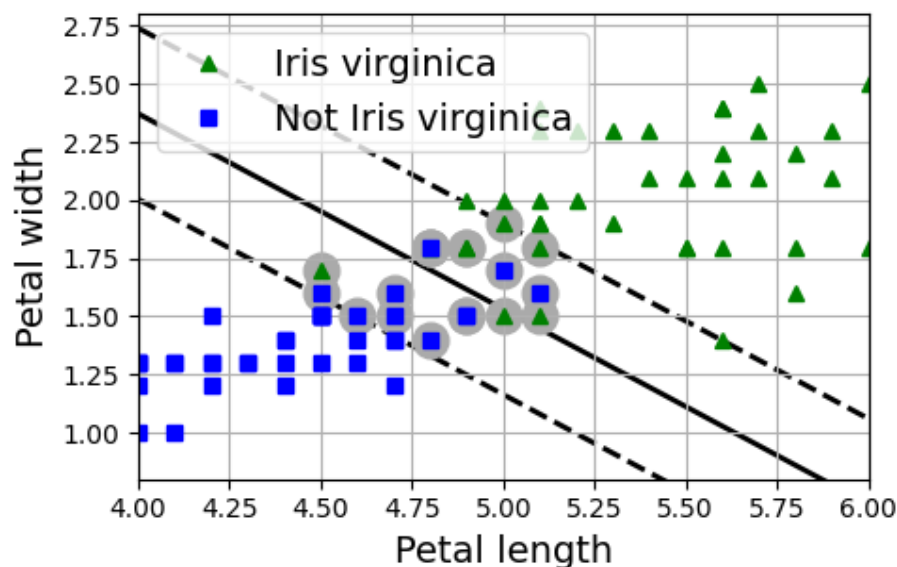
외우는 것(편견)은 지양합시다

서포트 벡터 머신(Support Vector Machine, SVM)

정의

▶ 마진(margin) 기반의 최적 함수를 찾는 지도학습 모델

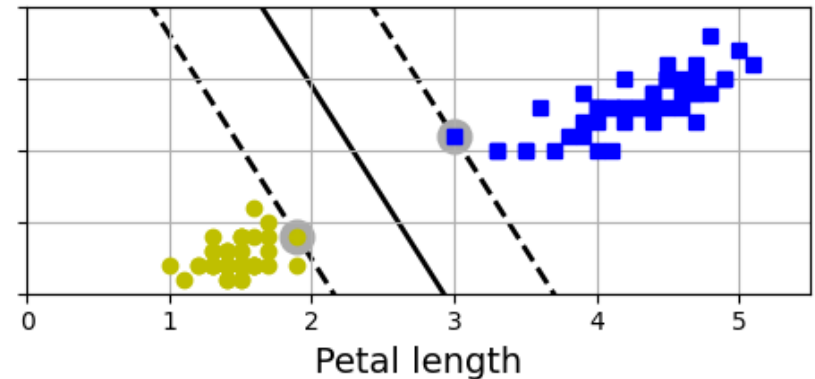
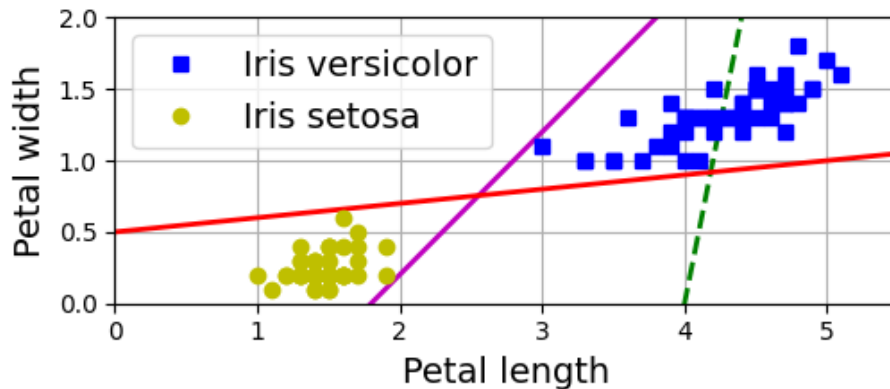
- 분류 문제: 클래스 간 마진을 최대화하는 **결정경계**를 찾음
- 회귀 문제: 허용 오차 범위 밖의 예측 오차를 최소화하는 **회귀함수**를 찾음



서포트 벡터 머신(Support Vector Machine, SVM)

SVM 분류기

- ▶ 두 클래스를 나누는 직선들 중 마진(margin)이 가장 큰 결정경계를 선택
 - Large margin classification 라고도 부름
 - 서포트 벡터(support vector): 결정경계와 가장 가까워 마진을 결정하는 샘플

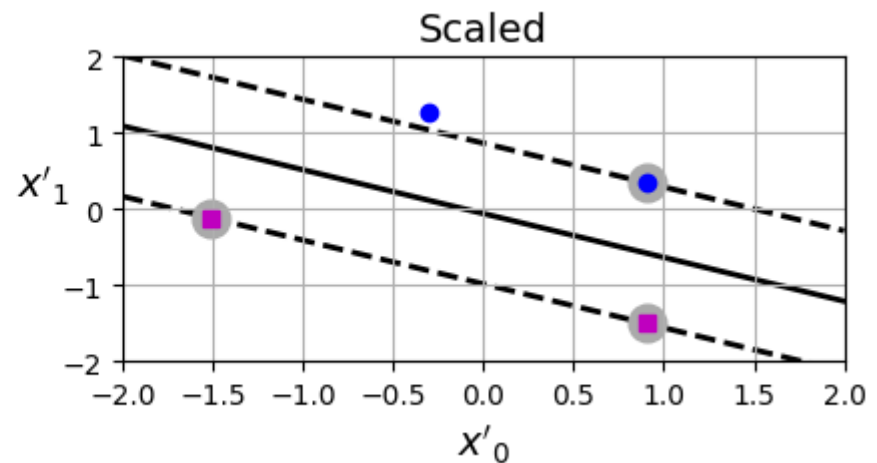
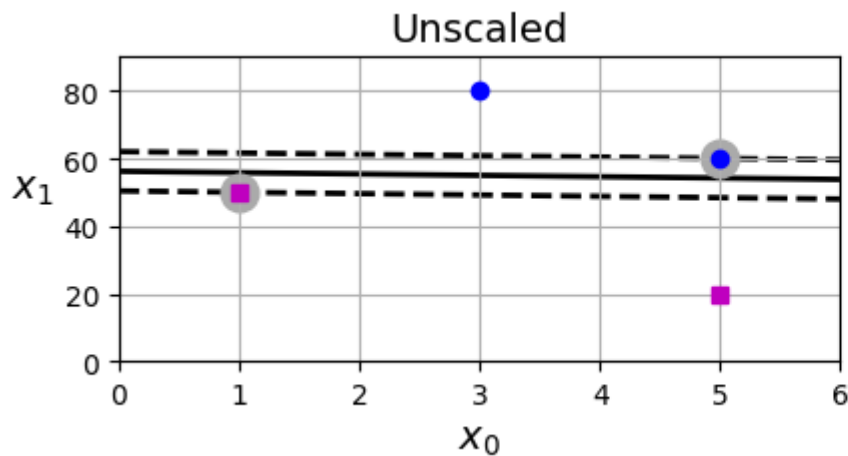


서포트 벡터 머신(Support Vector Machine, SVM)

특징 스케일링의 중요성

▶ 마진이 특징의 스케일(scale)에 민감하므로 SVM 분류 시 특징 스케일링이 필요

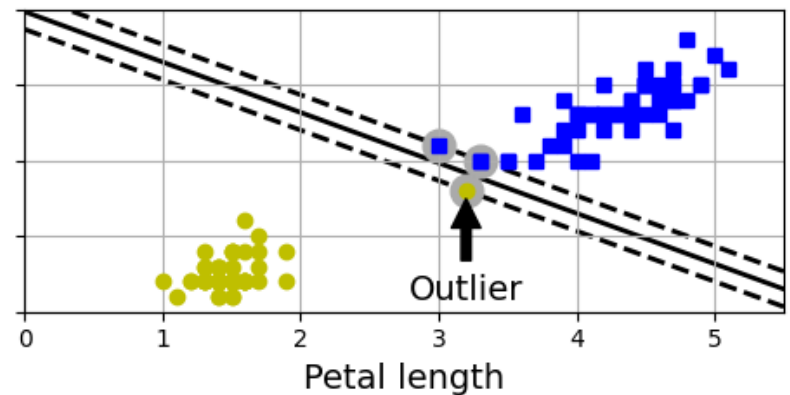
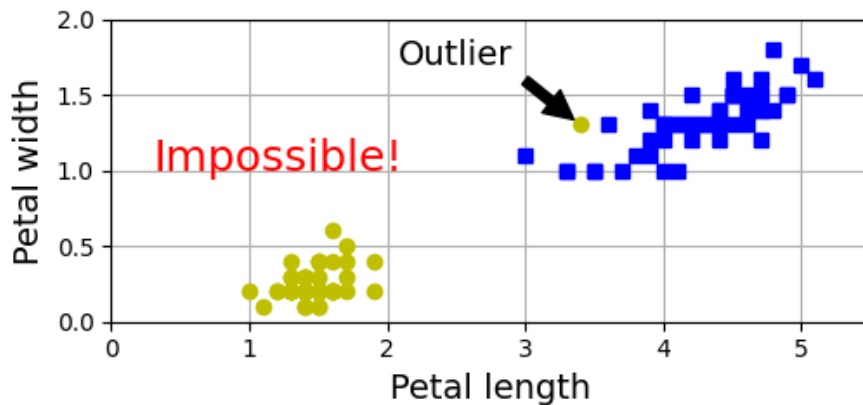
- 특징 스케일링 전: 마진 여유 없음
- 특징 스케일링 후: 마진 여유 확보



하드 마진 분류(Hard Margin Classification)

▶ 모든 샘플을 완벽히 분리하면서 마진을 최대화하는 분류

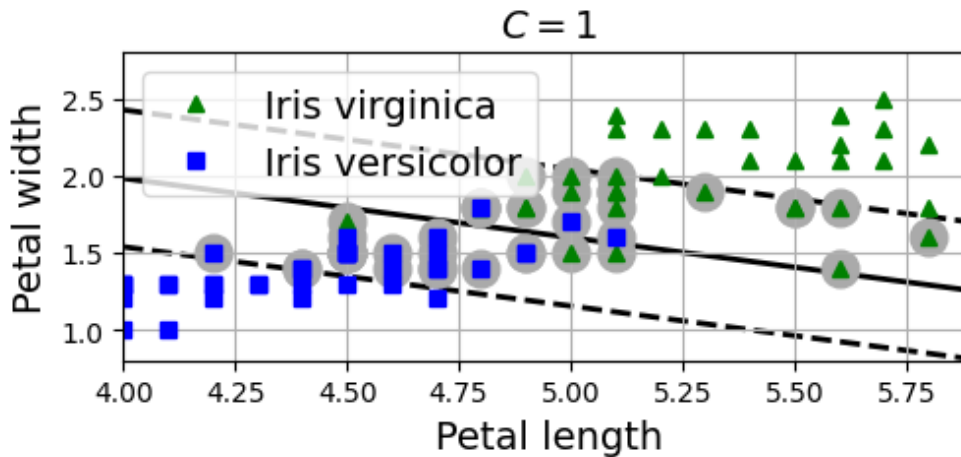
- 샘플 오분류(misclassification)를 허용하지 않음
- 한계 1: 데이터가 선형 분리 가능해야 함
- 한계 2: 이상치(outlier)에 민감함



■ 소프트 마진 분류(Soft Margin Classification)

▶ 마진 위반을 허용하면서, 가능한 큰 마진을 갖는 결정경계를 찾는 분류 방식

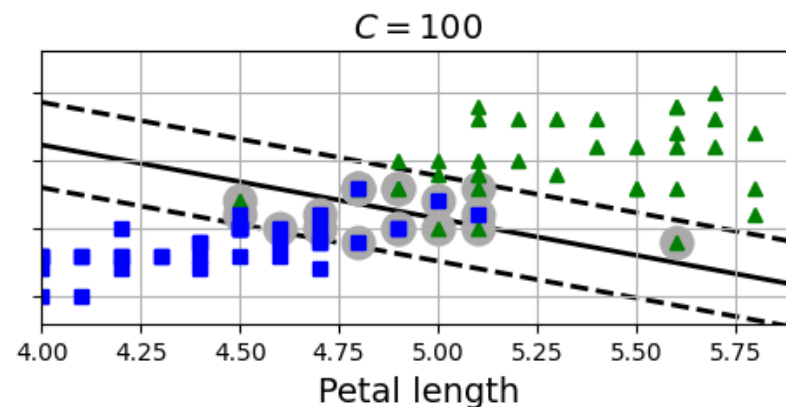
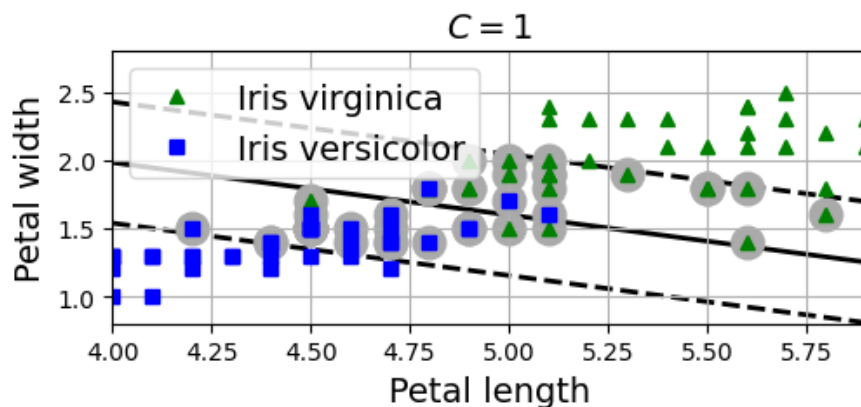
- 마진 위반(margin violation): 샘플이 경계 내부에 위치하거나 오분류된 경우
- 현실적인 데이터 분류에 적합



■ 규제 하이퍼파라미터(C)

▶ 마진 위반에 부여하는 벌점(penalty)의 크기를 조절하는 값

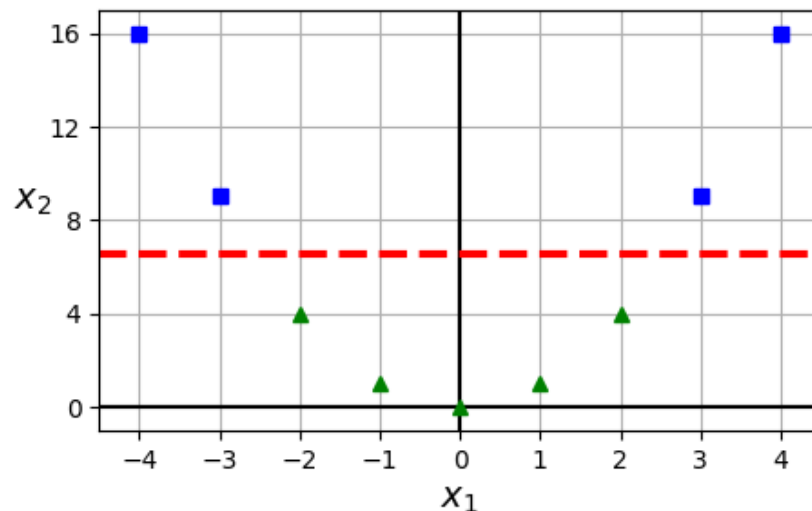
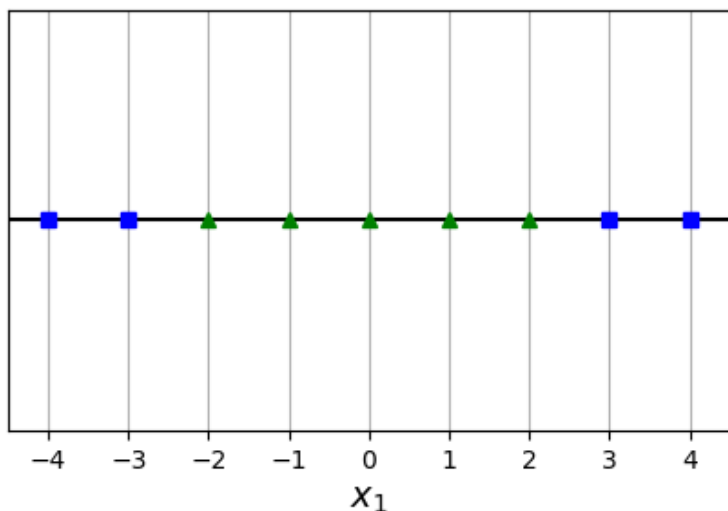
- $C \uparrow \rightarrow$ 마진 작아짐 $\rightarrow$ 마진 위반 감소 (overfitting 유의)
- $C \downarrow \rightarrow$ 마진 커짐 $\rightarrow$ 마진 위반 증가 (underfitting 유의)



■ 선형 SVM의 한계

▶ 선형 분류가 어려운 비선형 데이터에 한계가 있음

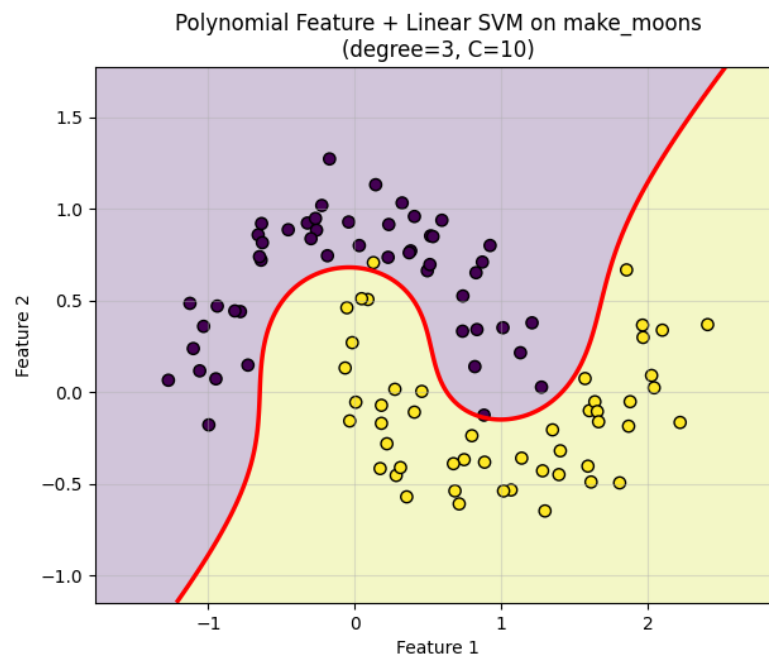
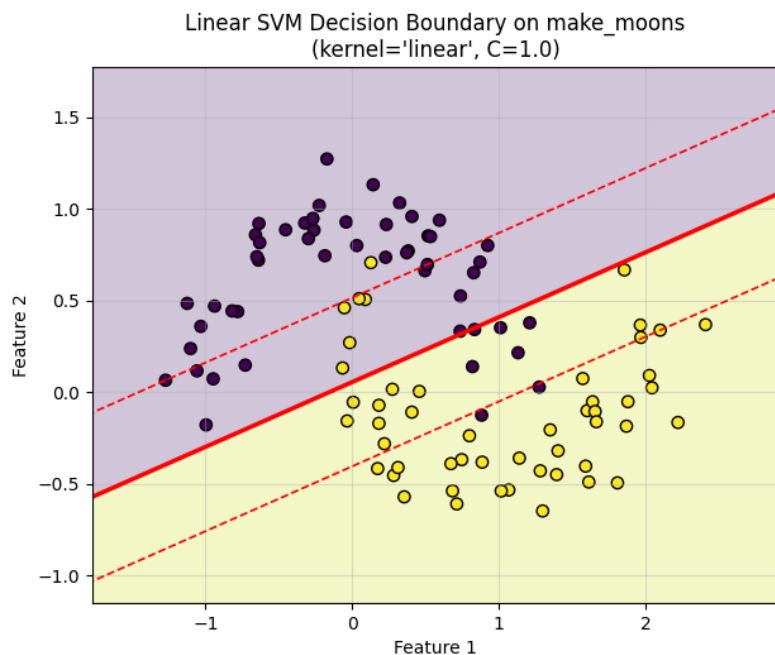
- 특징이 1개인 경우: 두 클래스가 섞여 있어 선형 분리가 어려움
- 특징이 2개인 경우: 더 높은 차원 공간(feature space)에서 선형 분리가 가능함
- 특징이 3개면? 4개면? n개면?



다항 특징(polynomial features) 기반 SVM 분류

▶ 다항 특징을 추가한 뒤, 확장된 특징 공간에서 선형 결정경계를 찾는 방법

- 기존 특징에 제곱항, 고차항, 교차항 등 추가
- 확장된 특징 공간에서 선형 SVM 적용 → 기존 공간에서 비선형 결정경계로 보임
- 차수 ↓ : 복잡한 데이터셋 분류 불가
- 차수 ↑ : 계산량 증가



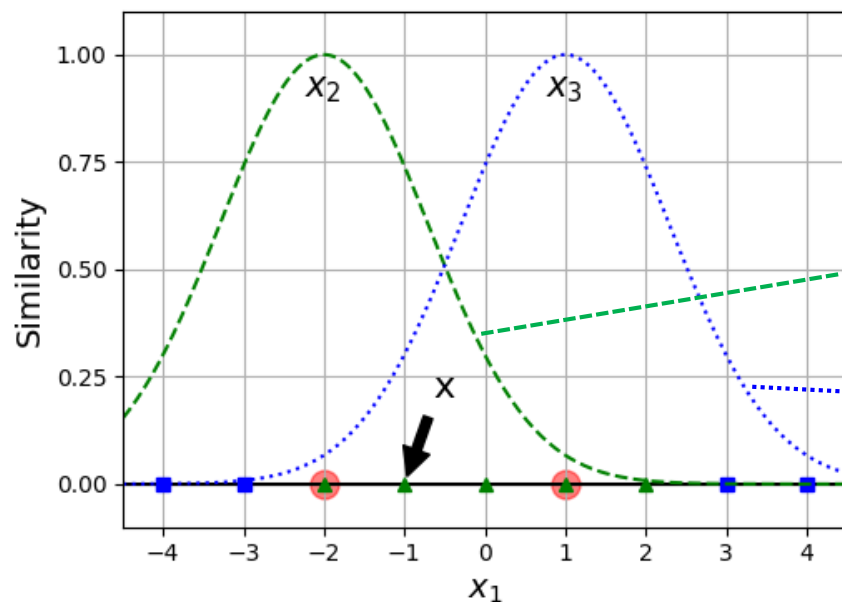
■ 유사도 특징(similarity feature) 기반 선형 SVM 분류(1)

▶ 랜드마크와의 유사도 특징을 추가한 공간에서 선형 결정경계를 찾는 방법

- 랜드마크(landmark): 각 샘플이 얼마나 가까운지 비교하기 위해 정해 둔 기준점
- 유사도 함수: 두 데이터 혹은 데이터와 랜드마크가 얼마나 가까운지 나타낸 함수

<방사 기저 함수(radial basis function, RBF)>

$$\phi(x, l) = \exp(-\gamma \|x - l\|^2)$$



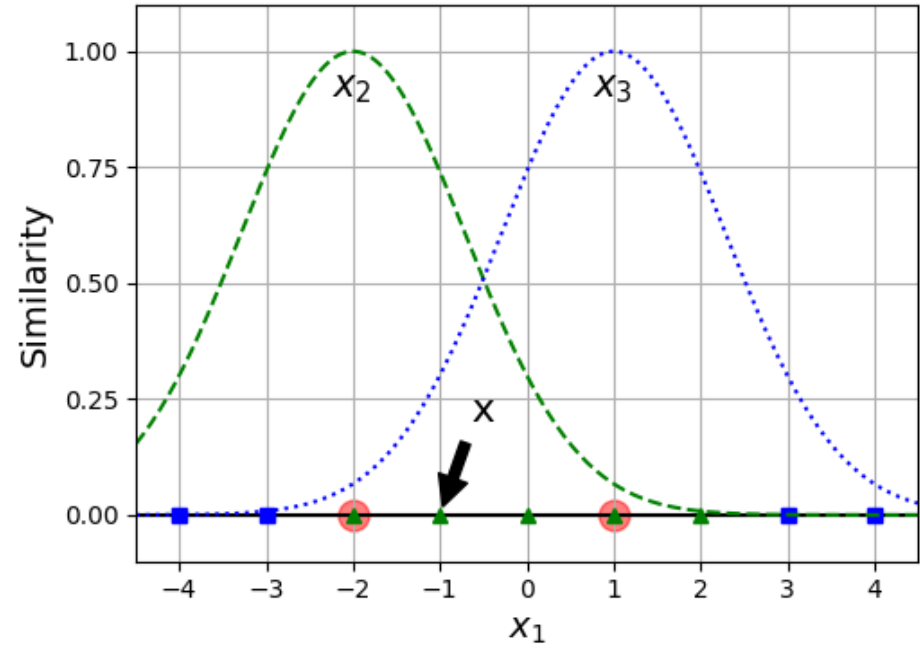
$$\gamma = 0.3, x_1 = -2, x_1 = 1$$

$$x_2 = \phi(x_1, -2) = \exp(-0.3\|x_1 + 2\|^2)$$

$$x_3 = \phi(x_1, 1) = \exp(-0.3\|x_1 - 1\|^2)$$

■ 유사도 특징(similarity feature) 기반 선형 SVM 분류(2)

▶ 유사도 함수(x_2, x_3) 계산

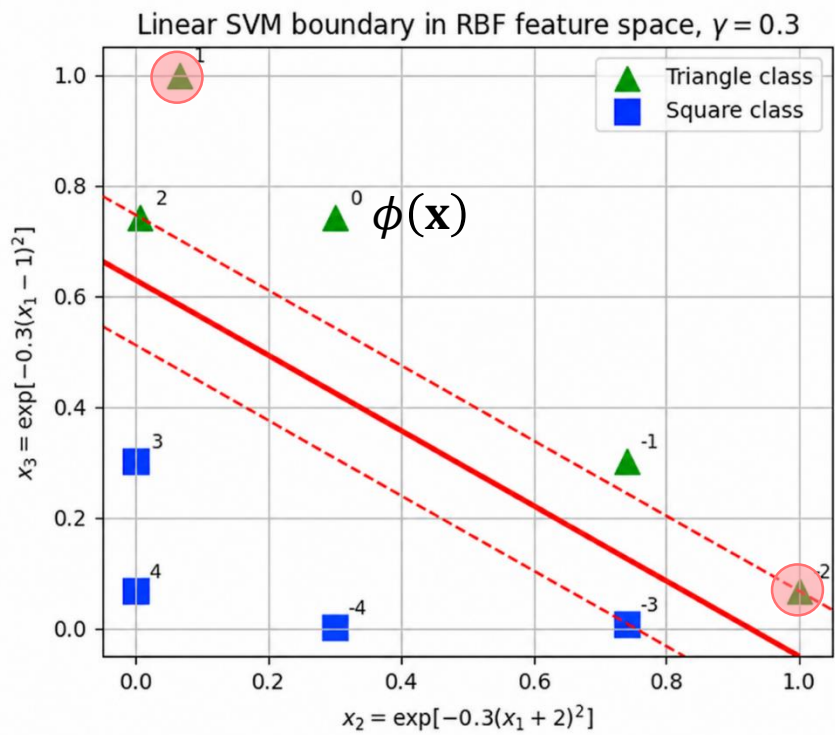


x_1	클래스	x_2	x_3
-4	■	0.301	0.001
-3	■	0.741	0.008
-2	▲	1.000	0.067
-1	▲	0.741	0.301
0	▲	0.301	0.741
1	▲	0.067	1.000
2	▲	0.008	0.741
3	■	0.001	0.301
4	■	0.000	0.067

유사도 특징(similarity feature) 기반 선형 SVM 분류(3)

유사도 함수(x_2, x_3) 계산 결과를 고차원 특징 공간(feature space)에 매핑

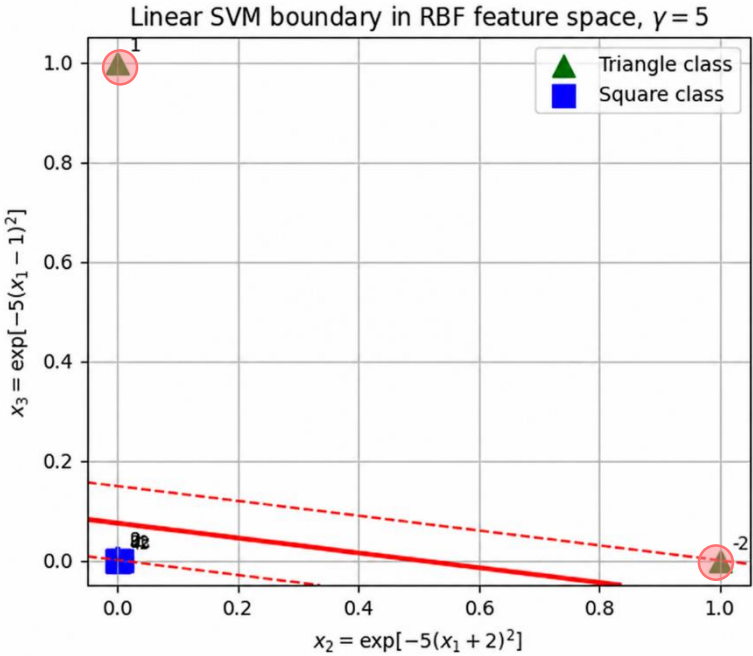
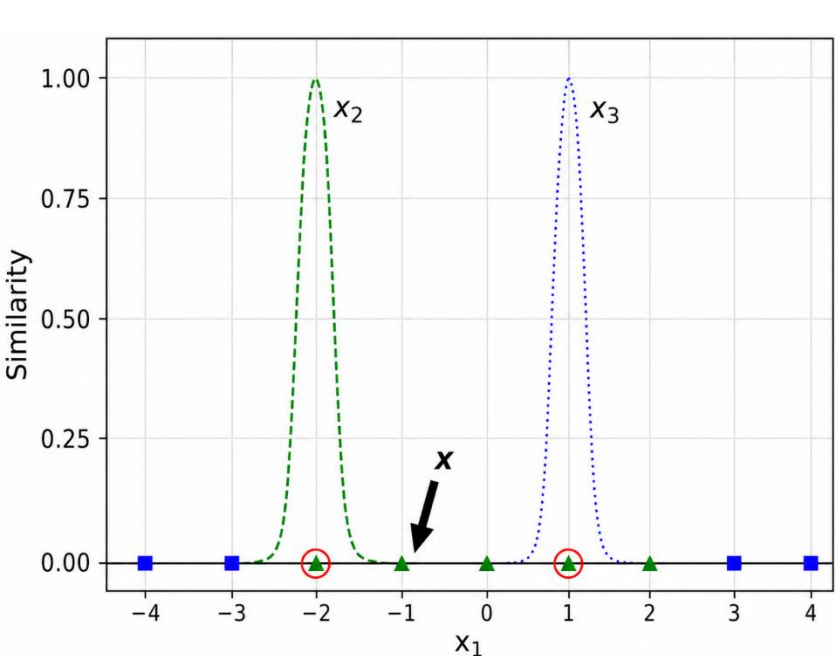
x_1	클래스	x_2	x_3
-4	■	0.301	0.001
-3	■	0.741	0.008
-2	▲	1.000	0.067
-1	▲	0.741	0.301
0	▲	0.301	0.741
1	▲	0.067	1.000
2	▲	0.008	0.741
3	■	0.001	0.301
4	■	0.000	0.067



■ 유사도 특징(similarity feature) 기반 선형 SVM 분류(4)

▶ 유사도 함수 형태에 따라 고차원 매핑 결과가 바뀜

- RBF 함수는 γ 가 클 경우, 각 랜드마크 주변에서만 강하게 반응하는 국소적인(local) 특징이 됨



■ 커널(Kernel)

▶ 두 샘플($\mathbf{x}, \mathbf{x}'$) 간의 유사도(similarity)를 계산하는 함수

■ 커널 트릭(Kernel Trick)

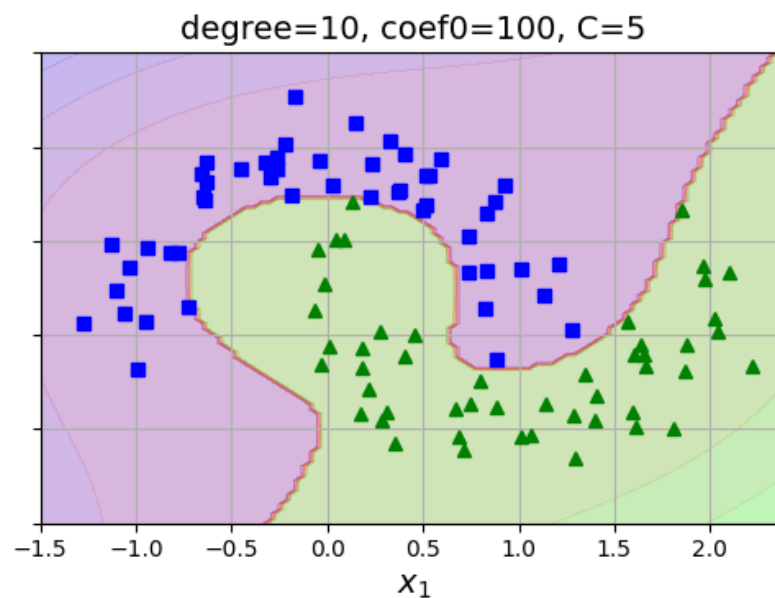
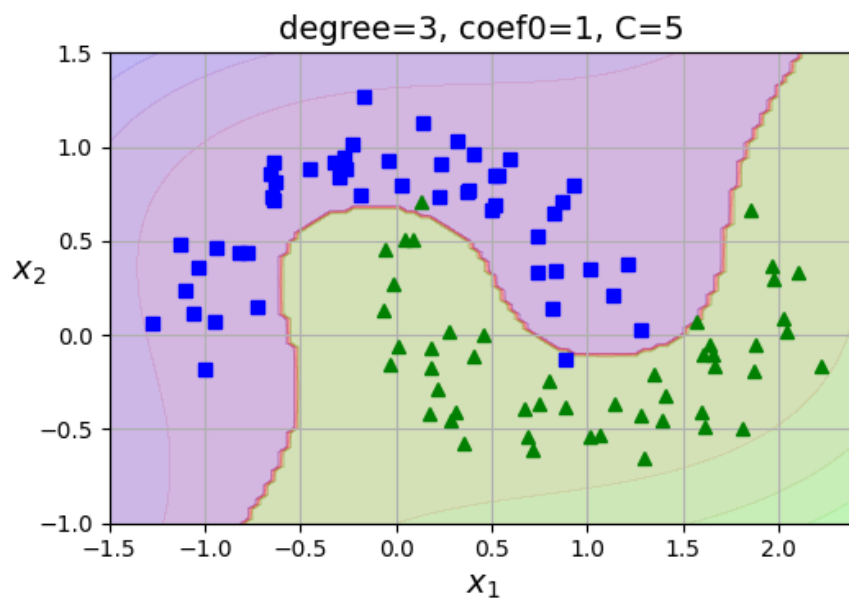
- ▶ 고차원 **특징을 추가하지 않고**, 커널 계산만으로 고차원 분류 효과를 얻는 방법
- 훈련 샘플을 변환할 필요 없으므로 계산량이 효율적

커널명	표현	비고
선형	$K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$	
다항식	$K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^T \mathbf{x}' + r)^d$	γ : 스케일링 인자, r : 상수, d : 차수
RBF(가우스)	$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	γ : 커널의 폭
시그모이드	$K(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^T \mathbf{x}' + r)$	γ : 스케일링 인자, r : 오프셋

다항식 커널

$$\blacktriangleright K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^T \mathbf{x}' + r)^d$$

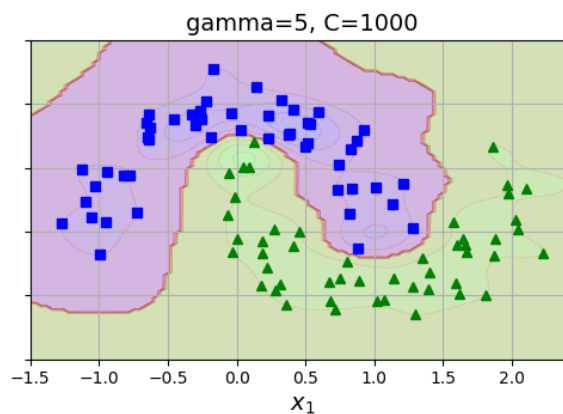
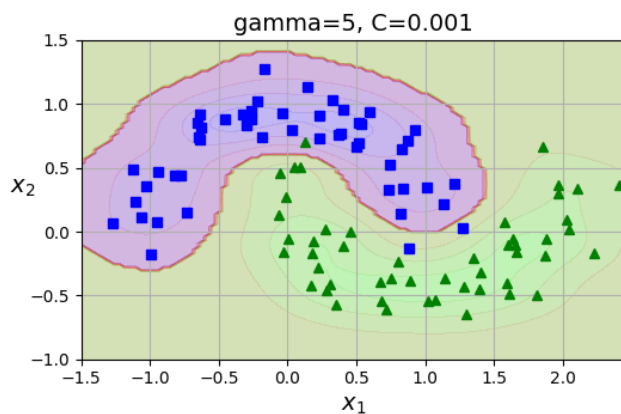
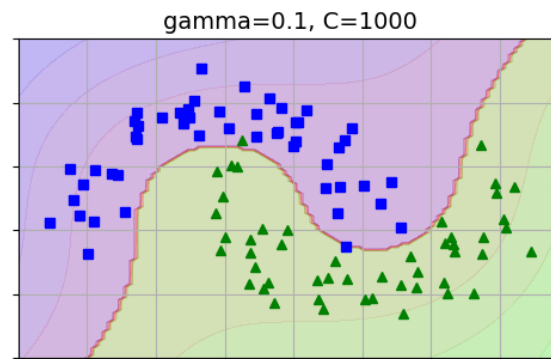
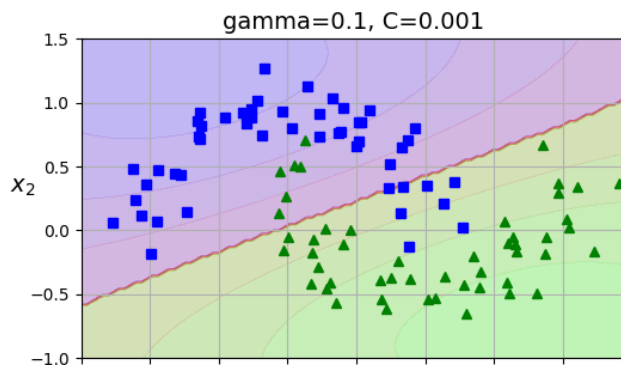
- γ : 스케일링 인자
- d : 차수(degree)
- r : 상수(높은 차수와 낮은 차수의 상대적 영향을 조절하여 결정경계 형태에 영향)



RBF (가우스) 커널

▶ $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$

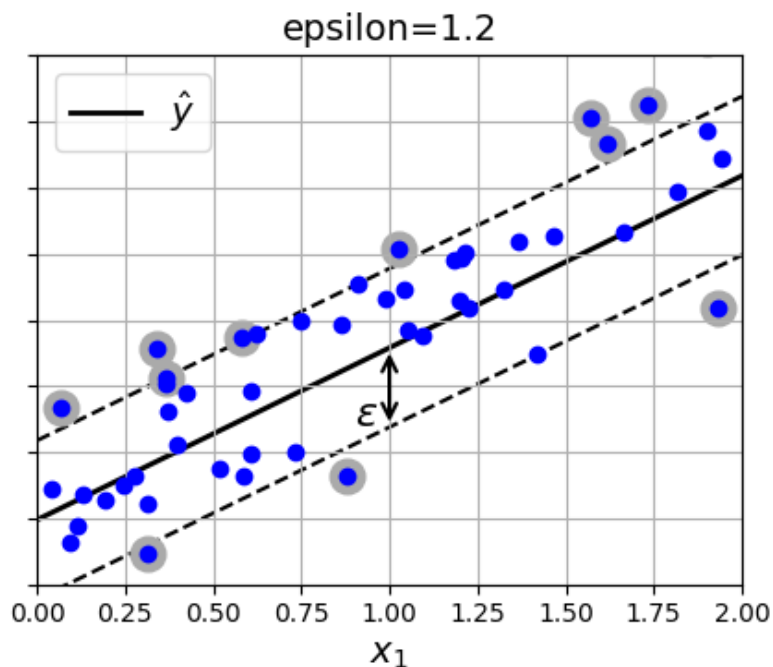
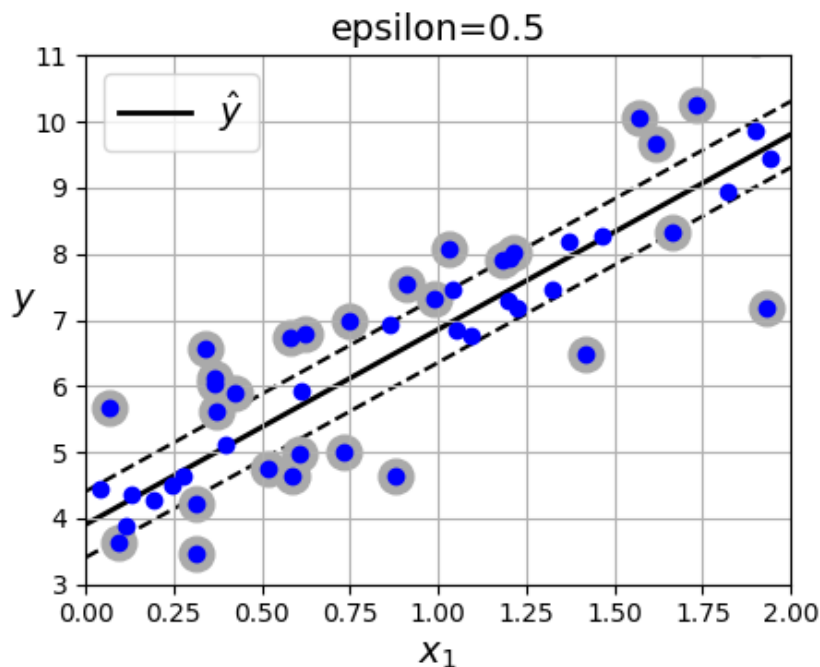
- γ : 커널의 폭(가우시안 분포의 분산의 역수)
- $\gamma \downarrow \rightarrow$ 분산이 크다(샘플이 넓은 범위에 영향) $\rightarrow$ 커널 폭이 넓음 $\rightarrow$ underfitting 위험
- $\gamma \uparrow \rightarrow$ 분산이 작다(샘플 주위에만 영향) $\rightarrow$ 커널 폭이 좁음 $\rightarrow$ overfitting 위험



개념

▶ 제한된 마진 오류(ε) 내에 가능한 많은 샘플이 들어가도록 학습하는 방법

- 목표 1: 마진 안에 최대한 많은 샘플을 포함($|y^{(i)} - \hat{y}^{(i)}| \leq \varepsilon$)
- 목표 2: 마진 밖의 예측값과 실제값 간의 오차를 최소화
- 마진 오류 $\downarrow \rightarrow$ 서포트 벡터 $\downarrow \rightarrow$ 모델 규제
- 마진 안에 훈련 샘플이 추가되어도 모델 예측에 영향 없음(ε -intensive)



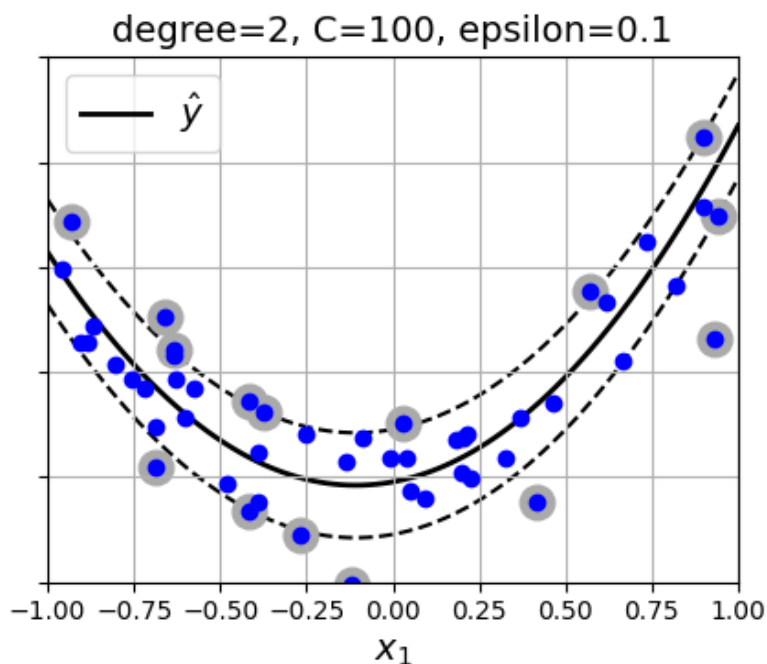
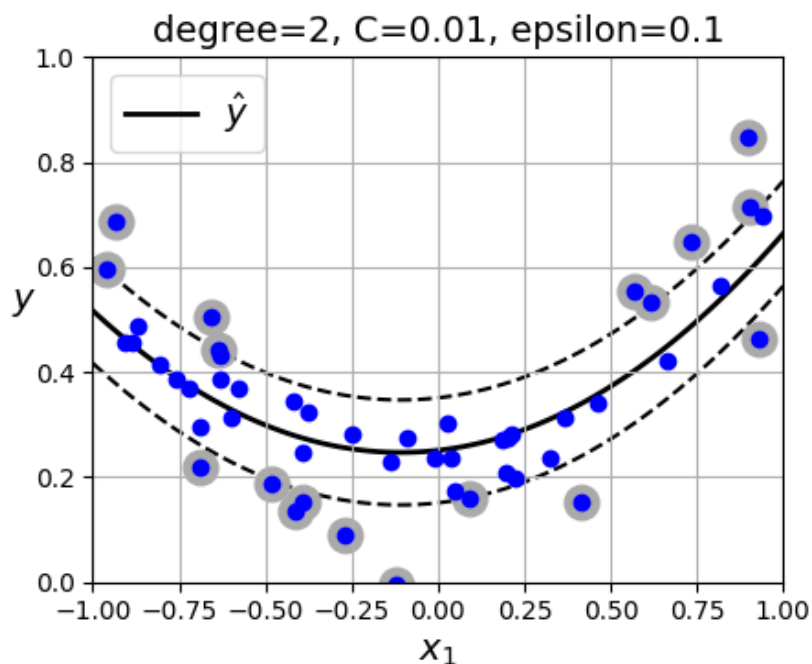
비선형 SVM 회귀

개념

▶ 커널(Kernel)을 이용해 비선형 관계를 표현하면서, ϵ 안의 오차는 무시하고 회귀

- 고차원 특징 공간에서 선형 SVM 회귀를 적용
- 기존 특징 공간에서 곡선 형태의 회귀모델 생성
- $C \downarrow \rightarrow$ 규제 증가

<2차 다항 커널 기반 비선형 SVM 회귀>



■ 규제(Regularization)

- ▶ 릿지 회귀
- ▶ 라쏘 회귀
- ▶ 엘라스틱 넷 회귀
- ▶ 규제 로지스틱 회귀
- ▶ 조기종료

■ 서포트 벡터 머신(SVM)

- ▶ 선형 SVM 분류
- ▶ 비선형 SVM 분류: 유사도 함수, 커널(Kernel)
- ▶ 선형/비선형 SVM 회귀

실습

■ 실습 준비

- ▶ LMS에 업로드 된 파일 다운로드
- ▶ 프로젝트 폴더 열기
- ▶ 가상환경 설정 확인

실습 1-1. 규제 선형 모델(1)

Ridge regression

▶ Ridge 클래스를 이용한 Ridge 회귀 수행

- `from sklearn.linear_model import Ridge`
- `Ridge(alpha=___)`
- `fit..^^`
- `predict..^^`

<참고>

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X} + \alpha \mathbf{A})^{-1} \mathbf{X}^T \mathbf{y}$$

▶ 확률적 경사 하강법을 사용한 Ridge 회귀 수행

- `from sklearn.linear_model import SGDRegressor`
- `SGDRegressor(penalty="l2", alpha=___/샘플수, tol=None, max_iter=___, eta0=___)`
- `fit..^^` # 참고: `fit()`은 1D 타깃을 기대(`.ravel()`: 넘파이 배열 1D 변환)
- `predict..^^`

<참고>

SGDRegressor: 확률적 경사하강법(SGD) 기반으로 훈련되는 선형 회귀 클래스
penalty: 규제종류, alpha: 규제 파라미터(규제시), tol: 수렴 조건, eta0: 학습률

Lasso regression

▶ Lasso 클래스를 이용한 Lasso 회귀 수행

- `from sklearn.linear_model import Lasso`
- `Lasso(alpha=___)`
- `fit..^^`
- `predict..^^`

▶ 확률적 경사 하강법을 사용한 Lasso 회귀 수행

- `from sklearn.linear_model import SGDRegressor`
- `SGDRegressor(penalty="l1", alpha=___, tol=None, max_iter=___, eta0=___)`
- `fit..^^`
- `predict..^^`

실습 1-3. 규제 선형 모델(3)

Elastic net regression

▶ ElasticNet 클래스를 이용한 Elastic net 회귀 수행

- `from sklearn.linear_model import ElasticNet`
- `ElasticNet(alpha=__, l1_ratio=__)`
- `fit..^^`
- `predict..^^`

— <엘라스틱 넷 회귀 비용 함수> —

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r \left(2\alpha \sum_{j=1}^m |\theta_j| \right) + (1 - r) \left(\frac{\alpha}{n} \sum_{j=1}^m \theta_j^2 \right)$$

▶ 조기 종료(Early Stopping)

▶ 훈련 / 검증 데이터셋 분리

▶ 전처리 파이프라인 구축(make_pipeline)

- PolynomialFeatures(degree=90, include_bias=False)
- StandardScaler()

▶ 특징 데이터에 전처리 적용

▶ SGD 기반 선형 회귀 모델 정의

- SGDRegressor(penalty=None, eta0=0.002, random_state=42)

▶ 반복문 내부에서 선형 회귀 모델을 점진적 학습

- partial_fit(전처리된 훈련 특징 데이터, 훈련 타깃 데이터)
- predict(전처리된 검증 특징 데이터)
- mean_squared_error(__, __, squared=False): 검증 오차 계산
- if문 작성: 지금까지 확인된 가장 낮은 RMSE보다 낮으면 에러값과 모델을 저장
※ 모델 저장은 from copy import deepcopy → 이후 deepcopy(모델) 수행

실습 3. 선형 SVM 분류

소프트 마진 분류

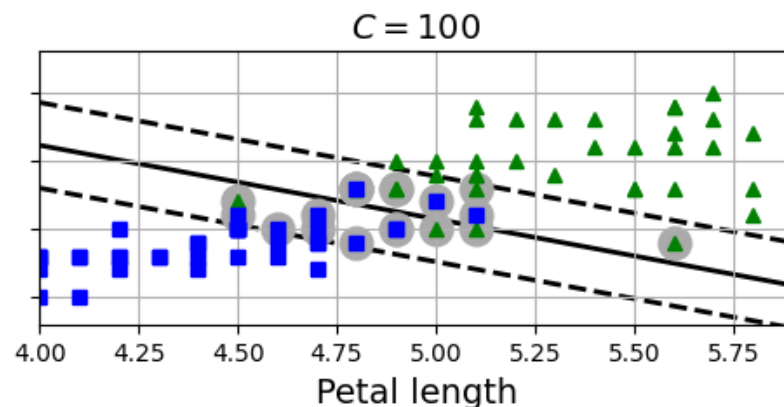
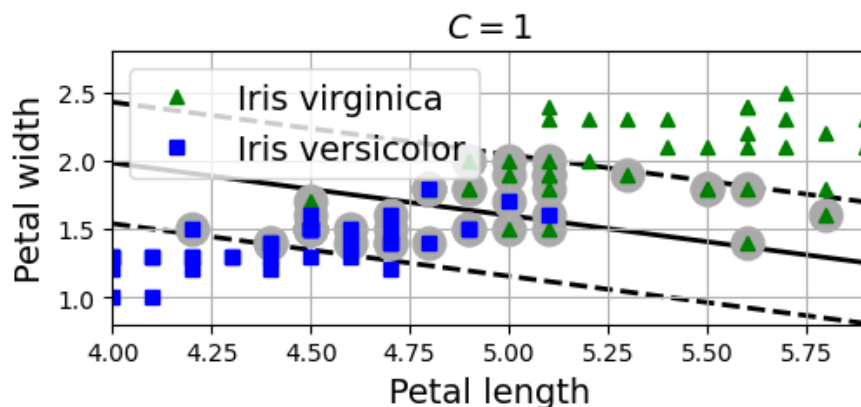
▶ LinearSVC 클래스를 이용한 선형 SVM 분류 수행

- `from sklearn.svm import LinearSVC`
- `LinearSVC(C=___, dual="auto", random_state=42)`
- `fit..^^`
- `predict..^^`

<참고>

LinearSVC: 클래스 확률을 추정하지 않음
predict는 예측 결과를 바로 도출 (점수 score는 추정 가능)

▶ 하이퍼파라미터 C를 바꾸면서 결과 확인



실습 4-1. 비선형 SVM 분류

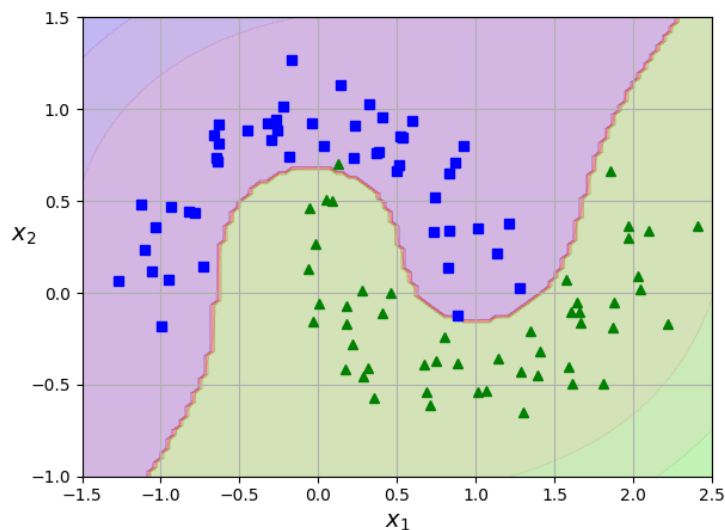
다항 특징(polynomial features) 기반 분류

▶ 전처리 및 모델 통합 파이프라인 구축(make_pipeline)

- 특징 공학(PolynomialFeatures 활용)
- 표준화
- 선형 SVM 분류기

▶ 파이프라인 fit()

▶ 하이퍼파라미터 C를 바꾸면서 결과 확인



실습 4-2. 비선형 SVM 분류

다항식 커널

▶ SVC 클래스의 다항식 커널을 사용한 SVM 분류기

- `from sklearn.svm import SVC`

▶ 전처리 및 모델 통합 파이프라인 구축 이후 `fit()`

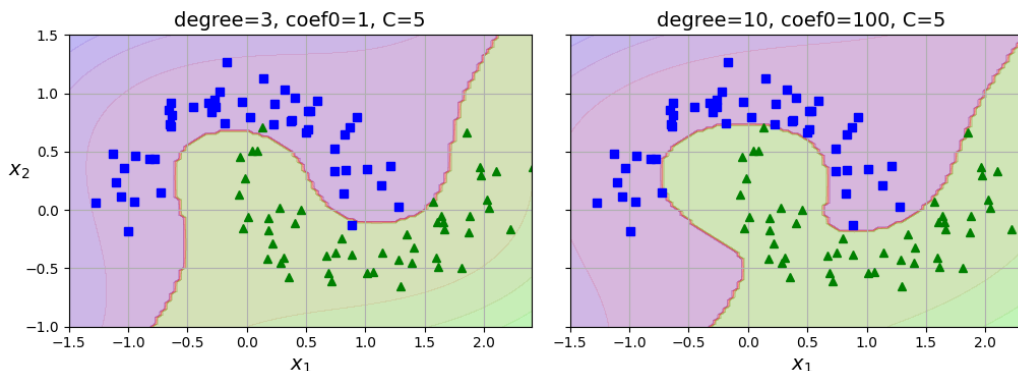
- 표준화
- `SVC(kernel="poly", degree=____, coef0=____, C=____)`

▶ 다항식 커널 하이퍼파라미터 변화에 따른 결과 확인

- 차수(d) 변화, (coef0=1, C=5 고정)
- 상수(coef0) 변화 (d=10, C=5 고정)

<참고>

$$K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^T \mathbf{x}' + r)^d$$



실습 4-3. 비선형 SVM 분류

▮ RBF(가우스) 커널

▶ 전처리 및 모델 통합 파이프라인 구축 이후 `fit()`

- 표준화
- `SVC(kernel="rbf", gamma=____, C=____)`

▶ RBF 커널 하이퍼파라미터 변화에 따른 결과 확인

- 감마(gamma) 변화 ($C=0.001$ 고정)
- 상수(C) 변화 ($\text{gamma}=5$ 고정)

실습 5-1. 선형 SVM 회귀

■ 선형 SVM 회귀

▶ 사이킷런 LinearSVR 클래스 호출

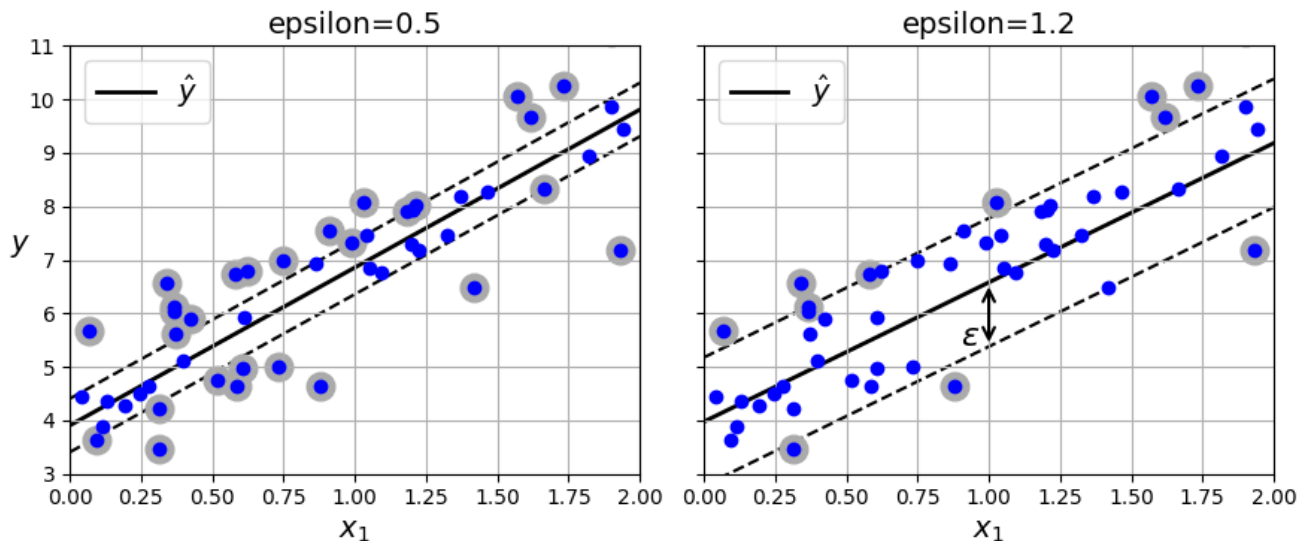
- `from sklearn.svm import LinearSVR`

▶ 전처리 및 모델 통합 파이프라인 구축 이후 `fit()`

- 표준화
- `LinearSVR(epsilon=__, dual="auto", random_state=42)`

▶ 하이퍼파라미터 변화에 따른 결과 확인

- `epsilon`을 바꾸며 결과 확인



실습 5-2. 비선형 SVM 회귀

■ 비선형 SVM 회귀

▶ 사이킷런 SVR 클래스 호출

- `from sklearn.svm import SVR`

▶ 전처리 및 모델 통합 파이프라인 구축 이후 `fit()`

- 표준화
- `SVR(kernel="poly", degree=__, C=__, epsilon=__)`

▶ 하이퍼파라미터 변화에 따른 결과 확인

- `degree`를 바꾸며 결과 확인(다른 하이퍼파라미터 고정)
- `epsilon`를 바꾸며 결과 확인(다른 하이퍼파라미터 고정)
- `C`를 바꾸며 결과 확인(다른 하이퍼파라미터 고정)

